

Pl/pgsql. Триггеры.

Лекция 9

Что такое pl/pgsql?

Это процедурный язык postgres, который может использоваться для создания функций, процедур и триггеров.

Основная идея: перенести логику из клиентского приложения на сервер СУБД

Поддерживает:

- переменные
- циклы и ветвления
- аргументы и возвращаемые значения
- существующие типы postgres
- SQL выражения

Зачем нужен pl/pgsql?

Основная причина: повышение производительности.

- Исключаются дополнительные обращения между клиентом и сервером
- Промежуточные ненужные результаты не передаются между сервером и клиентом
- Есть возможность избежать многочисленных разборов одного запроса
- Можно создавать триггеры и курсоры

Недостатки pl/pgsql

- Нагрузка на СУБД и ограничение в масштабировании
- Сложности с контролем версий
- Сложности с отладкой
- Проблемы с миграцией на другую платформу
- Устаревшие подходы процедурного программирования
- Слабые проверки корректности кода
- Сложности с юнит тестированием

Нужны веские причины, чтобы переносить логику на pl/pgsql, но иногда это необходимо!

Hello World

```
DO $$ -- анонимный блок кода, выполняется сразу
DECLARE -- объявления переменных, не обязательно
    hello TEXT := 'HELLO';
    world TEXT := 'WORLD';
BEGIN -- ниже идет код
    RAISE NOTICE '% %', hello, world;
END $$;
```

Создание функции

```
CREATE OR REPLACE FUNCTION get_library_name()  
RETURNS TEXT AS $$  
BEGIN  
    RETURN 'Городская библиотека';  
END;  
$$ LANGUAGE plpgsql;
```

-- Вызов функции

```
SELECT get_library_name();
```

-- Удаление функции

```
DROP FUNCTION IF EXISTS get_library_name;
```

Список функций

```
SELECT
```

```
    n.nspname AS schema,  
    p.proname AS function_name,  
    pg_get_function_arguments(p.oid) AS arguments,  
    pg_get_functiondef(p.oid) AS definition
```

```
FROM pg_proc p
```

```
JOIN pg_namespace n ON p.pronamespace = n.oid
```

```
WHERE n.nspname = 'public' -- иначе будут и функции из pg_catalog
```

```
    AND p.prokind = 'f' -- только функции (не процедуры)
```

```
ORDER BY p.proname;
```

Базовый синтаксис

```
CREATE OR REPLACE FUNCTION function_name(param_name param_type)
RETURNS return_type AS $$
DECLARE -- это начало блока
    variable_name variable_type; -- объявление переменных
BEGIN
    -- тело функции
    -- SELECT ... INTO variable_name FROM ...;
    RETURN result;
END; -- это конец блока
$$ LANGUAGE plpgsql;
```


Функция с запросом

```
CREATE OR REPLACE FUNCTION get_book_title(p_book_id INTEGER)
RETURNS TEXT AS $$
DECLARE
    v_title TEXT;    -- переменная для хранения результата, NULL по-умолчанию
BEGIN
    -- Выбираем значение в переменную
    SELECT title INTO v_title
    FROM books
    WHERE book_id = p_book_id;

    RETURN v_title;
END;
$$ LANGUAGE plpgsql;

SELECT get_book_title(1); -- Вернет "Война и мир"
```

Подстановка переменных

```
-- SELECT с подстановкой переменной (p_loan_id)
SELECT  bl.due_date INTO v_current_due_date FROM book_loans bl
JOIN readers r ON bl.reader_id = r.reader_id
WHERE bl.loan_id = p_loan_id;  -- ← переменная подставляется напрямую!

-- Вычисление новой даты
v_new_due_date := v_current_due_date + p_extra_days;

-- UPDATE с подстановкой переменных
UPDATE book_loans
SET due_date = v_new_due_date  -- ← переменная!
WHERE loan_id = p_loan_id;    -- ← переменная!
```

Функция с несколькими параметрами

```
CREATE OR REPLACE FUNCTION calculate_due_date(  
    p_loan_date DATE,  
    p_loan_days INTEGER  
)  
RETURNS DATE AS $$  
BEGIN  
    RETURN p_loan_date + p_loan_days;  
END;  
$$ LANGUAGE plpgsql;
```

OUT параметры

```
CREATE OR REPLACE FUNCTION get_book_stats(  
    p_book_id INTEGER, -- можно написать IN перед параметром (по-умолчанию)  
    OUT total_copies INTEGER, -- проинициализированы NULL  
    OUT available_copies INTEGER,  
    OUT on_loan INTEGER  
) AS $$ BEGIN  
    SELECT quantity, available_quantity  
    INTO total_copies, available_copies  
    FROM books  
    WHERE book_id = p_book_id;  
  
    on_loan := total_copies - available_copies;  
END;  
$$ LANGUAGE plpgsql;
```

	☐ total_copies ▾	☐ available_copies ▾	☐ on_loan ▾
1	12	2	10

Обращение к отдельным полям результата

```
SELECT
```

```
    stats.total_copies,  
    stats.available_copies,  
    stats.on_loan
```

```
FROM get_book_stats(1) AS stats;
```

Результат тот же, что и при

```
SELECT * FROM get_book_stats(1);
```

А если выполним:

```
SELECT get_book_stats(1);
```

☐ get_book_stats	▼	⌵
(12,2,10)		

INOUT параметры

```
CREATE OR REPLACE FUNCTION normalize_loan_days(  
    INOUT loan_days INTEGER,  
    OUT was_adjusted BOOLEAN,  
    OUT adjustment_reason TEXT  
) AS $$ BEGIN  
    was_adjusted := FALSE;  
    adjustment_reason := NULL;  
  
    IF loan_days < 7 THEN  
        loan_days := 7;  
        was_adjusted := TRUE;  
        adjustment_reason := 'Минимальный срок выдачи - 7 дней';  
    ELSIF loan_days > 30 THEN  
        loan_days := 30;  
        was_adjusted := TRUE;  
        adjustment_reason := 'Максимальный срок выдачи - 30 дней';  
    END IF;  
END; $$ LANGUAGE plpgsql;  
  
SELECT * FROM normalize_loan_days(5); -- применение
```

	☐ loan_days ▾	÷	☐ was_adjusted ▾	÷	☐ adjustment_reason ▾	÷
1	7	•	true		Минимальный срок выдачи - 7 дней	

Тип RECORD

```
CREATE OR REPLACE FUNCTION get_reader_info(p_reader_id INTEGER)
RETURNS RECORD AS $$
DECLARE
    result RECORD;
BEGIN
    SELECT
        r.first_name || ' ' || r.last_name AS full_name,
        r.email,
        r.registration_date,
        COUNT(bl.loan_id) AS total_loans,
        COUNT(bl.loan_id) FILTER (WHERE bl.status = 'active') AS active_loans
    INTO result FROM readers r
    LEFT JOIN book_loans bl ON r.reader_id = bl.reader_id
    WHERE r.reader_id = p_reader_id
    GROUP BY r.reader_id, r.first_name, r.last_name, r.email, r.registration_date;

    RETURN result;
END;
$$ LANGUAGE plpgsql;
```

Использование RECORD

Нужно указывать структуру явно при вызове

```
SELECT * FROM get_reader_info(1) AS (  
    full_name TEXT,  
    email varchar(150),  
    registration_date DATE,  
    total_loans BIGINT,  
    active_loans BIGINT  
);
```


RETURNS TABLE

```
CREATE OR REPLACE FUNCTION get_overdue_books()
RETURNS TABLE (
    reader_name TEXT,
    book_title VARCHAR(255),
    days_overdue INTEGER
) AS $$ BEGIN
    RETURN QUERY -- не прекращает выполнение, а добавляет данные в результат
    SELECT
        r.last_name || ' ' || r.first_name,
        b.title,
        (CURRENT_DATE - bl.due_date)::INTEGER
    FROM book_loans bl
    JOIN books b ON bl.book_id = b.book_id
    JOIN readers r ON bl.reader_id = r.reader_id
    WHERE bl.status = 'overdue' ORDER BY bl.due_date;
END;
$$ LANGUAGE plpgsql;
```

IF - ELSIF - ELSE

```
SELECT COUNT(*), BOOL_OR(status = 'overdue')  
INTO v_active_loans, v_has_overdue FROM book_loans  
WHERE reader_id = p_reader_id AND status = 'active';
```

```
IF v_has_overdue THEN  
    can_borrow := false; -- присвоение  
    reason := 'У вас есть просроченные книги';  
ELSIF v_active_loans >= 5 THEN  
    can_borrow := false;  
    reason := 'Превышен лимит книг на руках (5)';  
ELSE  
    can_borrow := true;  
    reason := 'Можно взять книгу';  
END IF;
```

CASE - WHEN

```
SELECT COUNT(*) INTO v_total_loans
FROM book_loans WHERE reader_id = p_reader_id;
```

```
v_category := CASE
    WHEN v_total_loans = 0 THEN 'Новичок'
    WHEN v_total_loans BETWEEN 1 AND 5 THEN 'Читатель'
    WHEN v_total_loans BETWEEN 6 AND 15 THEN 'Активный читатель'
    ELSE 'Завсегдатай библиотеки'
END;
```

postgresql цикл по полям



Найти

postgresql цикл по полям синий трактор едет к нам

postgresql цикл по полям синий трактор едет к нам у него в прицепе кто

Цикл LOOP (безусловный)

[<<метка>>]

LOOP

операторы

END LOOP [метка];

EXIT [метка] [WHEN логическое-выражение]; – заканчивает выполнение **внутреннего** цикла. Если существует метка, то заканчивает выполнение цикла с меткой.

CONTINUE [метка] [WHEN логическое-выражение]; – продолжает выполнение **внутреннего** цикла. Если существует метка, то продолжает выполнение цикла с меткой

Цикл LOOP (пример)

```
v_x := 1;
<<outer_loop>>  -- Метка внешнего цикла
LOOP
  EXIT outer_loop WHEN v_x > 5;  -- Выход из внешнего
  v_y := 1;
  <<inner_loop>>  -- Метка внутреннего цикла
  LOOP
    EXIT inner_loop WHEN v_y > 5;  -- Выход из внутреннего
    -- Пропускаем диагональ
    IF v_x = v_y THEN
      v_y := v_y + 1;
      CONTINUE inner_loop;
    END IF;
    -- Останавливаемся если сумма > 8
    EXIT outer_loop WHEN v_x + v_y > 8;
    -- Возвращаем строку
    RETURN QUERY SELECT v_x, v_y, v_x + v_y;
    v_y := v_y + 1;
  END LOOP inner_loop;
  v_x := v_x + 1;
END LOOP outer_loop;
```

Результат

--	x		y		sum
--		----	+	----	-----
--	1		2		3
--	1		3		4
--	1		4		5
--	1		5		6
--	2		1		3
--	2		3		5
--	2		4		6
--	2		5		7
--	3		1		4
--	3		2		5
--	3		4		7
--	3		5		8
--	4		1		5
--	4		2		6
--	4		3		7

Циклы WHILE и FOR

[<<метка>>]

WHILE логическое-выражение LOOP

операторы

END LOOP [метка];

[<<метка>>]

FOR имя IN [REVERSE] выражение .. выражение [BY выражение] LOOP

операторы

END LOOP [метка];

```
FOR i IN REVERSE 10..1 BY 2 LOOP
```

```
-- внутри цикла переменная i будет иметь значения 10,8,6,4,2
```

```
END LOOP;
```

Отладка и ошибки

RAISE [уровень] 'формат' [, выражение [, ...]] – где уровень DEBUG, LOG, INFO, NOTICE, WARNING и EXCEPTION. Только EXCEPTION вызывают ошибку, остальные могут записываться в лог и отправляться клиенту. По-умолчанию, NOTICE и выше отправляется клиенту.

Для отладки можно использовать уровень NOTICE

```
RAISE NOTICE 'Начало выдачи: reader_id=%, book_id=%', p_reader_id, p_book_id;
```

А вот это уже будет ошибка и выполнение будет прервано

```
RAISE EXCEPTION 'Читатель с ID % не найден', p_reader_id;
```


Обработка ошибок

BEGIN

операторы

EXCEPTION

WHEN **условие** [OR **условие** ...] THEN

операторы_обработчика

END;

BEGIN

UPDATE mytab SET **firstname** = 'Joe' WHERE **lastname** = 'Jones';

x := **x** + 1;

y := **x** / 0;

EXCEPTION

WHEN **division_by_zero** THEN

RAISE NOTICE '**перехватили ошибку division_by_zero**';

RETURN **x**;

END;

<https://postgrespro.ru/docs/enterprise/18/errcodes-appendix>

Вложенность блоков

```
<< outerblock >>
```

```
DECLARE
```

```
    quantity integer := 30;
```

```
BEGIN
```

```
    RAISE NOTICE 'Сейчас quantity = %', quantity;  -- Выводится 30
```

```
    quantity := 50;
```

```
    -- Вложенный блок
```

```
    DECLARE
```

```
        quantity integer := 80;
```

```
    BEGIN
```

```
        RAISE NOTICE 'Сейчас quantity = %', quantity;  -- Выводится 80
```

```
        RAISE NOTICE 'Во внешнем блоке quantity = %', outerblock.quantity;  --
```

```
Выводится 50
```

```
    END;
```

```
    RAISE NOTICE 'Сейчас quantity = %', quantity;  -- Выводится 50
```

```
    RETURN quantity;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

Процедуры

Отличаются от функций тем, что:

- создаются с помощью команды `CREATE PROCEDURE`
- не возвращают результат явно с помощью `RETURNS`, но могут выдать данные с помощью `OUT` параметров
- не могут использоваться в `DML`, а вызываются специальной командой `CALL`
- поддерживают механизм транзакций внутри
 - вызов процедуры открывает транзакцию
 - после завершения транзакции (через `commit` или `rollback`), новая транзакция начинается автоматически

```
CREATE OR REPLACE PROCEDURE return_book(
    p_loan_id INTEGER
) AS $$
DECLARE
    v_book_id INTEGER;
BEGIN
    SELECT book_id INTO v_book_id FROM book_loans
    WHERE loan_id = p_loan_id AND status = 'active';
    IF v_book_id IS NULL THEN
        RAISE EXCEPTION 'Активная выдача % не найдена', p_loan_id;
    END IF;
    UPDATE book_loans SET status = 'returned',
        return_date = CURRENT_DATE WHERE loan_id = p_loan_id;

    UPDATE books SET available_quantity = available_quantity + 1
    WHERE book_id = v_book_id;
    COMMIT; -- ПРОЦЕДУРА может делать COMMIT!
END;
$$ LANGUAGE plpgsql;
```

Управление процедурами

```
CALL return_book(11); -- вызов процедуры
```

```
DROP PROCEDURE return_book; -- удаление процедуры
```

```
SELECT -- выведем список процедур
```

```
    n.nspname AS schema_name,
```

```
    p.proname AS procedure_name
```

```
FROM pg_catalog.pg_proc p
```

```
JOIN pg_catalog.pg_namespace n ON n.oid = p.pronamespace
```

```
WHERE
```

```
    p.prokind = 'p' -- 'p' stands for procedure
```

```
    AND n.nspname NOT IN ('pg_catalog', 'information_schema')
```

```
ORDER BY schema_name, procedure_name;
```

Функции vs Процедуры

Используем функцию, когда

- Возвращаем данные (SELECT)
- Используем в запросах с WHERE или JOIN
- Создаем триггеры

Используем процедуру, когда

- Сложная бизнес логика в транзакциях
- Батчевая обработка данных с периодическими коммитами
- Административные задачи

Триггеры

Что такое триггеры?

Триггер – это функция, которая запускается автоматически при определенном событии, пользовательском и системном.

Виды пользовательских событий:

- BEFORE INSERT/UPDATE/DELETE/TRUNCATE – выполняются перед операцией
- AFTER INSERT/UPDATE/DELETE/TRUNCATE – выполняются после операции
- INSTEAD OF INSERT/UPDATE/DELETE/TRUNCATE – выполняются вместо операции, но применяются только для представлений

Задумывались ли вы когда-нибудь о том, можно ли вставлять/обновлять данные в представлении?

Зачем нужны триггеры?

- Аудит и логирование (логирование изменения статуса)
- Обновление вычисляемых полей (при выдаче книги уменьшить quantity)
- Валидация и бизнес логика (запрет выдачи, если есть просрочки)
- Каскадные операции (при удалении читателя архивируем выдачи)
- Уведомления и интеграции (при просрочке отправить уведомление)
- Выполнение операций без необходимости выдавать права (пишем лог в недоступной схеме)

Типы триггеров

- Уровня строк (FOR EACH ROW) – запускаются для каждой строки
- Уровня операторов (FOR EACH STATEMENT) – запускаются один раз для операции

Если UPDATE затронул 100 строк, то триггерная функция вызовется 100 раз.

Триггеры BEFORE уровня строк позволяют менять или отменять результат операции.

Триггеры для события TRUNCATE могут быть только уровня оператора.

Порядок выполнения триггеров

1. **BEFORE STATEMENT** (до выполнения операции для всего запроса)
2. **BEFORE ROW** (до обработки каждой строки)
3. **Операция** (вставка, изменение или удаление данных)
4. **AFTER ROW** (после обработки каждой строки)
5. **AFTER STATEMENT** (после выполнения операции для всего запроса)

В случае нескольких триггеров на одно событие, триггеры выполняются в алфавитном порядке их названия (например, триггер `trigger_a` выполнится раньше `trigger_b`)

Создание триггеров

1. Сначала создается триггерная функция
2. Затем создается триггер, используя триггерную функцию

Мы можем создать несколько триггеров, используя одну функцию

Триггерная функция, это функция, которая

- не имеет аргументов
- возвращает специальный тип **TRIGGER**
- может использовать специальные переменные **NEW, OLD, TG_***

Переменные триггерной функции

- NEW – новая строка базы данных для INSERT/UPDATE
- OLD – старая строка для UPDATE/DELETE
- TG_NAME – имя триггера
- TG_WHEN – BEFORE, AFTER или INSTEAD OF
- TG_LEVEL – ROW или STATEMENT
- TG_OP – INSERT, UPDATE, DELETE или TRUNCATE
- TG_TABLE_NAME – имя таблицы
- TG_TABLE_NAME – имя схемы

<https://postgrespro.ru/docs/postgresql/18/plpgsql-trigger>

Пример триггерной функции

```
CREATE OR REPLACE FUNCTION decrease_book_quantity()  
RETURNS TRIGGER AS $$  
BEGIN  
    -- NEW - это новая строка, которую INSERT/UPDATE пытается вставить  
    -- Уменьшаем доступное количество на 1  
    UPDATE books  
    SET available_quantity = available_quantity - 1  
    WHERE book_id = NEW.book_id;  
  
    -- AFTER триггер игнорирует возвращаемой значение  
    RETURN NEW; -- можно вернуть NULL  
END; $$ LANGUAGE plpgsql;
```

Пример создания триггера

```
CREATE TRIGGER trg_book_loan_decrease_quantity
AFTER INSERT ON book_loans          -- После вставки в book_loans
FOR EACH ROW                        -- На каждую строку
EXECUTE FUNCTION decrease_book_quantity();
```

Пример триггерной функции

```
CREATE OR REPLACE FUNCTION increase_book_quantity()
RETURNS TRIGGER AS $$ -- возвращает TRIGGER
BEGIN
    -- NEW.status - новое значение статуса
    -- OLD.status - старое значение статуса

    -- Если книгу вернули (статус изменился на 'returned')
    IF NEW.status = 'returned' AND OLD.status != 'returned' THEN
        UPDATE books
        SET available_quantity = available_quantity + 1
        WHERE book_id = NEW.book_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```


Пример создания триггера

```
CREATE TRIGGER trg_book_loan_increase_quantity  
AFTER UPDATE ON book_loans  
FOR EACH ROW  
EXECUTE FUNCTION increase_book_quantity() ;
```

ИЛИ

```
CREATE OR REPLACE TRIGGER trg_book_loan_increase_quantity  
AFTER UPDATE of status ON book_loans -- мы можем указать колонку  
FOR EACH ROW  
EXECUTE FUNCTION increase_book_quantity() ;
```

Триггер с WHEN

```
CREATE OR REPLACE FUNCTION set_default_due_date()  
RETURNS TRIGGER AS $$  
BEGIN  
    -- NEW.due_date уже NULL (проверим в триггере)  
    NEW.due_date := NEW.loan_date + INTERVAL '14 days';  
    --  Возвращаем измененный NEW - он будет вставлен!  
    RETURN NEW;  
END; $$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_set_due_date  
BEFORE INSERT ON book_loans  
FOR EACH ROW  
WHEN (NEW.due_date IS NULL) -- Сработает только если due_date не указан!  
EXECUTE FUNCTION set_default_due_date();
```

Возвращаемые значения в триггерных функциях

Для триггеров уровня операторов нужно всегда возвращать NULL

Для триггеров AFTER возвращаемое значение игнорируется, можно вернуть NULL

Для триггеров BEFORE:

Событие	NEW	OLD	Return
INSERT	✓	✗	NEW или измененную строку или NULL для пропуска операции
UPDATE	✓	✓	NEW или измененную строку или NULL для пропуска операции
DELETE	✗	✓	OLD или NULL для пропуска операции

Просмотр и удаление триггеров

```
SELECT -- просмотр всех триггеров
    event_object_table AS table_name,
    trigger_name,
    event_manipulation AS event,
    action_statement AS definition,
    action_timing AS activation
FROM
    information_schema.triggers
ORDER BY
    table_name,
    trigger_name;
```

```
DROP TRIGGER trg_set_due_date ON book_loans; -- триггер всегда привязан к
таблице, поэтому указываем при удалении
```

SECURITY INVOKER vs SECURITY DEFINER

По-умолчанию функция создается в режиме SECURITY INVOKER, т.е. выполняется с правами той роли, которая вызвала функцию. Внутри работают все правила доступа для проверки допустимости операций.

Если мы хотим получить доступ до данных, доступ к которым ограничен для пользовательской роли, то можно создать функцию в режиме SECURITY DEFINER. Это позволит вызывать функцию с правами роли, которая является владельцем функции.

SECURITY DEFINER пример

```
-- создали защищенную схему (доступ только для админов)
CREATE SCHEMA IF NOT EXISTS secure_audit;

CREATE TABLE secure_audit.book_changes_log (
    -- ...
);

REVOKE ALL ON SCHEMA secure_audit FROM PUBLIC;
REVOKE ALL ON TABLE secure_audit.book_changes_log FROM PUBLIC;
```

SECURITY DEFINER пример

```
CREATE OR REPLACE FUNCTION log_book_changes()  
RETURNS TRIGGER  
SECURITY DEFINER  -- ⚠ КЛЮЧЕВОЙ МОМЕНТ! функция выполняется с правами  
владельца  
SET search_path = secure_audit, public  -- 🔒 Безопасность: фиксируем  
search_path  
AS $$  
BEGIN  
    -- Эта вставка выполнится с правами владельца функции,  
    -- даже если у текущего пользователя нет прав на secure_audit!  
    INSERT INTO book_changes_log (  
        -- ...  
    );  
  
    RETURN NEW;  
END; $$ LANGUAGE plpgsql;
```